Strategy Pattern

✅ What is it?

> The **Strategy Pattern** lets you **define multiple algorithms** (strategies) and **select one at runtime**.

It helps you **separate the algorithm from the object that uses it**, so that:

- you can **change behavior dynamically**
- avoid `if-else` or `switch-case` code

📦 Real-world analogy:

> You have a `PaymentService`, and you want to support:

- `CreditCardPayment`
- `PaypalPayment`
- `ApplePayPayment`

Instead of doing:

```
if (type.equals("card")) {...}
else if (type.equals("paypal")) {...}
```

➡️ You define a `PaymentStrategy` interface, and each implementation handles its own logic.

```java
public class Strategy {  no usages
    interface SortStrategy{  5 usages  2 implementations
        List<Product> sort(List<Product> products);  1 usage  2 implementations
    }

    static class SortByPrice implements SortStrategy{  no usages
        @Override  1 usage
        public List<Product> sort(List<Product> products) {
            return products.stream()
                    .sorted(Comparator.comparing(Product::getPrice))
                    .toList();
        }
    }
    static class SortByDate implements SortStrategy{  no usages
        @Override  1 usage
        public List<Product> sort(List<Product> products) {
            return  products.stream()
                    .sorted(Comparator.comparing(Product::getDate))
                    .toList();
        }
    }
```

```java
static class ProductSorterService{  no usages
    private final Map<String, SortStrategy> strategies;  2 usages

    ProductSorterService(Map<String, SortStrategy> strategies) {  no usages
        this.strategies = strategies;
    }

    public List<Product> processSort(String method,List<Product> products) {  no usages
        SortStrategy strategy=strategies.get(method);
        if (strategy == null) throw new IllegalArgumentException("Unknown method: " + method);
        return strategy.sort(products);
    }
}
```

Template Method Pattern

✅ Définition :

> Le **Template Method Pattern** définit **un squelette d'algorithme** dans une **classe abstraite**,
> tout en **laissant certaines étapes personnalisables** dans des **sous-classes**.

☕ Imagine you're making tea and coffee

Steps are:

1. Boil water
2. Brew drink
3. Pour into cup
4. Add condiments

But…

- **Boiling water, pouring** → always the same ☑️
- **Brewing and condiments** → depend on drink ❓

```java
abstract class CaffeineBeverage {
    public final void prepareRecipe() {
        boilWater();
        brew();           // depends on drink
        pourInCup();
        addCondiments();  // depends on drink
    }

    private void boilWater() {
        System.out.println("Boiling water");
    }

    private void pourInCup() {
        System.out.println("Pouring into cup");
    }

    // 👇 Child classes must implement
    protected abstract void brew();
    protected abstract void addCondiments();
}
```

Now for tea :

```java
class Tea extends CaffeineBeverage {
    protected void brew() {
        System.out.println("Steeping the tea");
    }

    protected void addCondiments() {
        System.out.println("Adding lemon");
    }
}
```

Concept	Meaning
`prepareRecipe()`	Template method (fixed sequence)
`final`	Prevents subclasses from changing the order
`brew()`, `addCondiments()`	Abstract steps (to be defined by subclasses)

Template Method is when you define the **main algorithm once**, but **allow parts to change** in subclasses.

Command Pattern

✅ Définition :

> Encapsuler une action (commande) dans une classe — pour pouvoir l'exécuter **plus tard**, **enregistrer** ce qu'on a fait, ou **changer dynamiquement ce qu'un bouton fait.**

Cela te permet :

- de **séparer celui qui demande l'action** de celui qui l'exécute
- de **garder un historique** (undo/redo)
- de **composer des files de commandes**

🤖 Exemple imagé : La télécommande d'une maison connectée

Tu as :

- Une **télécommande** → elle ne connaît pas les appareils (lumière, porte...)
- Tu appuies sur un **bouton A** → ça exécute une **commande**
- Cette commande appelle la **vraie lumière** et lui dit "allume-toi"

🧱 Structure (en version simplifiée) :

Élément	Rôle
`Command`	interface avec une méthode `execute()`
`LightOnCommand`	classe concrète qui exécute `light.turnOn()`
`Light`	l'objet réel (Receiver)
`RemoteControl`	la télécommande qui déclenche `execute()`

on a besoin :

1️⃣ Une interface Command
2️⃣ Des classes concrètes qui implémentent Command
3️⃣ Une classe "télécommande" (invoker)

```java
public class CommandDP {
    //1. Interface de commande
    interface Command{
        void execute();
        void undo();
    }
    //2. Appareil (la lumière)
    static class Light {
        public void turnOn() {
            System.out.println("💡 La lumière est allumée");
        }

        public void turnOff() {
            System.out.println("💡 La lumière est éteinte");
        }
    }
    // 3. Commande concrète (Light ON)
    static class LightOnCommand implements Command {
        private final Light light;

        public LightOnCommand(Light light) {
            this.light = light;
        }
        @Override
        public void execute() {
            light.turnOn();  // ⬅️ on appelle l'objet réel
        }
        public void undo() {
            light.turnOff();  // ⬅️ pour annuler, on éteint
        }
    }
    //4. La télécommande (Invoker)
    static class RemoteControl {
        private Command currentCommand;
        private Command lastCommand; // pour undo

        public void setCommand(Command command) {
            this.currentCommand = command;
        }

        public void pressButton() {
            currentCommand.execute();
            lastCommand = currentCommand;
        }

        public void pressUndo() {
            if (lastCommand != null) {
```

```
                lastCommand.undo();  // ⬅️ on annule la dernière
action
            }
        }
    }
    public static void main(String[] args) {
        Light light = new Light();
        Command lightOn = new LightOnCommand(light);

        RemoteControl remote = new RemoteControl();
        remote.setCommand(lightOn);

        remote.pressButton();    // 💡 Light is ON
        remote.pressUndo();      // 💡 Light is OFF
    }
}
```

✅ Qu'est-ce que c'est ?

C'est un pattern où **plusieurs objets** (les "handlers") se passent une requête
les uns après les autres,
jusqu'à ce que l'un d'eux la traite — ou pas du tout.

📦 Cas d'usage typique :

- Pipeline de validation (`valider email → valider mot de passe → valider droits`)
- Filtres HTTP (ex: sécurité, cache…)
- Traitement d'un événement (si handler A ne sait pas, il passe à B)

✅ C'est ça Chain of Responsibility :

👉 Tu as plusieurs handlers (= agents)

👉 Chaque handler peut :

- **traiter** la demande
- ou dire : "je passe au suivant"

Chain of Responsibility

✅ Qu’est-ce que c’est ?

> C’est un pattern où **plusieurs objets** (les "handlers") se passent une requête
> **les uns après les autres**,
> **jusqu’à ce que l’un d’eux la traite** — ou pas du tout.

📦 Cas d’usage typique :

- Pipeline de validation (`valider email → valider mot de passe → valider droits`)
- Filtres HTTP (ex: sécurité, cache…)
- Traitement d’un événement (si handler A ne sait pas, il passe à B)